

WHAT THIS SESSION IS

Run on 12 September 2026, and it is two sessions in one. The first eighty minutes finish middleware with two examples that matter in real products: stripping personal data out of a message before it ever reaches a hosted model, and giving the agent a state of its own beyond the message list. After the break Akshay Agarwal takes over and starts MCP, the Model Context Protocol. No MCP code is written in this half, on purpose. It answers the question you should ask first: we can already write tools and call APIs, so why does another protocol need to exist? It ends inside the protocol itself, on JSON-RPC 2.0, which session 8 picks up.

CONTENTS

1. Where this session picks up
2. Personal data, and why it should not reach the model
3. PII Middleware and its four strategies
4. Teaching it what an Indian phone number looks like
5. Blocking outright, and handling the error
6. Custom state: when messages are not enough
7. A middleware class that updates the state
8. Middleware, in one page
9. MCP: the problem before the protocol
10. Why not just wrap every API in a tool
11. What MCP actually changes
12. The architecture: host, client, server
13. The three primitives: tools, resources, prompts
14. The standard methods every server speaks
15. The data layer: JSON-RPC 2.0
16. Clarifications from the Q&A
17. Key takeaways

1

Where this session picks up

Last session covered middleware across seven files: the four hooks, a dynamic prompt, dynamic tools, dynamic model routing, and two built-ins for retries and summarisation. This one opens with a quick recap and a new analogy, then adds two more examples before moving on.

THE AIRPORT, INSTEAD OF THE SECURITY GUARD

Think of the agent as the aeroplane. Its job is to fly. Middleware is the airport around it: check-in, security, boarding gates, customs on arrival. None of those change how the plane flies. They decide what happens before and after, and they can be stricter for some passengers than for others. Whether you use a built-in middleware or write your own with a hook, the plane still flies the same way.

The recap in class also listed the hooks. Worth adding from the official documentation: besides the four covered last week, LangChain also has `before_agent` and `after_agent`, which run once at the very start and very end of a whole agent run rather than around each model call. They were named in the room but not used.

Reference: docs.langchain.com/oss/python/langchain/middleware

2 Personal data, and why it should not reach the model

PII stands for personally identifiable information: phone numbers, email addresses, card numbers, anything that points at a real person.

Picture an ordinary support chatbot. It asks for your phone number and you type it, without a second thought, because that is what people do. Now remember what happens next. The agent calls the model, and most of the time that model is a closed model on somebody else's servers. Your customer's phone number has just left your company.

WITHOUT PII MIDDLEWARE

```
"call me on 9876543210,          to OpenAI / Anthropic / Google
card 4111 1111 1111 1111" -----> the real number, the real card
                                   on a third party's servers
```

WITH PII MIDDLEWARE

```
"call me on 9876543210,      +-----+
card 4111 1111 1111 1111" -| cleaned  |-> "call me on [REDACTED_PHONE],
                           | here     |   card ****-****-****-1111"
                           +-----+

                                   the model never sees
                                   the real values
```

WHY THIS IS A COMPLIANCE ISSUE, NOT A NICETY

For many companies this is a regulatory problem, not a preference. Personal data leaving your boundary is exactly what data protection rules care about. The observation made in class was simple: a phone number or an email address is useless to the model for answering the question. It does not need them, so do not send them.

3 PIIMiddleware and its four strategies

PIIMiddleware is built in. No hook, no function of your own. You tell it which kind of data to look for and what to do when it finds it.

pii-middleware.py

```
import re
import sys

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMatch, PIIMiddleware, PIIDetectionError

load_dotenv()
sys.stdout.reconfigure(encoding="utf-8")

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[],
    system_prompt="You are a support agent. Confirm what the customer told you in one line.",
    middleware=[
        PIIMiddleware("email", strategy="redact"),
        PIIMiddleware("credit_card", strategy="mask"),
        PIIMiddleware("phone", detector=indian_phone, strategy="redact"),
    ],
)

message = (
    "Hi, I am Asha. Mail me at asha.k@example.com or call 9876543210. "
    "I paid with card 4111 1111 1111 1111."
)

result = agent.invoke({"messages": [{"role": "user", "content": message}]})

print("What customer typed:")
print(" ", message)
print()
print("What the model received:")
print(" ", result["messages"][0].content)
print()
print("Answer:")
print(" ", result["messages"][-1].content)
```

One middleware per kind of data, stacked in the list. Email, credit card, IP address, MAC address and URL have detectors built in. Phone numbers do not, for a good reason covered in the next section.

The four strategies

Strategy	What happens	Use it when
redact	The value is replaced with a label, such as [REDACTED_EMAIL]	The model does not need the value at all
mask	Most of it is hidden, a small hint stays, usually the last four digits	A person may need to recognise which card it was, the way a banking app shows ****1111
hash	The value is replaced with a stable hash, the same input always gives the same hash	You need to match or group later without storing the real value
block	The request is stopped. It never reaches the model, and an error is raised	This data must never be in the conversation at all

Run live, the customer's line went in with the real email, number and card, and the message the model received had a label where the email and phone were and a masked card ending in the last four digits.

NOT EVERY APPLICATION NEEDS THIS

Asked in the room, and the answer was the right one. If the data is not sensitive for your use case, or you are running a local model where nothing leaves your servers, you may not need it. Middleware is chosen per use case. Nothing here is compulsory.

Reference: docs.langchain.com/oss/python/langchain/middleware/built-in

4 Teaching it what an Indian phone number looks like

There is no built-in phone detector because there is no single phone format. India, the UAE, the United States and Australia all write numbers differently. So you write the detector yourself.

```
def indian_phone(text: str) -> list[PIIMatch]:
    """Find 10 digit Indian mobile numbers, with or without the country code."""
    matches = []
    for found in re.finditer(r"(?:\+91[\s-]?)?[6-9]\d{9}", text):
        matches.append(
            PIIMatch(type="phone", value=found.group(),
                    start=found.start(), end=found.end())
        )
    return matches
```

Reading the pattern piece by piece

(?:\+91[\s-]?)?	[6-9]	\d{9}
-----	-----	-----
		+++ nine more digits, any digit
	+++ the first digit of an Indian mobile	
	number is always 6, 7, 8 or 9	
+++ optionally: +91, then optionally one space or hyphen		
the whole group is optional, so 9876543210 matches too		


```
matches: 9876543210
         +919876543210
         +91 9876543210
         +91-9876543210
```

What the detector hands back

- It receives the raw text and returns a list of `PIIMatch` objects. No matches means an empty list.
- `type` is the label, used in the redacted text.
- `value` is the exact text that matched.
- `start` and `end` are the character positions. These matter most: the middleware needs to know exactly where to cut the string so it can replace that slice and nothing else.

DO NOT GET STUCK ON THE REGULAR EXPRESSION

Said plainly in class: the syntax of the pattern is not the lesson. You can write a better detector with different logic and it will work, as long as it returns matches in this shape. The lesson is that built-in middleware accepts your own detection logic when the built-in one does not fit your data.

5 Blocking outright, and handling the error

Switching the credit card strategy to `block` stopped the run with an error before anything reached the model. That is the intended behaviour, and several students asked how to handle it gracefully instead of letting the program crash.

```
strict = create_agent(  
    model="openai:gpt-4o-mini",  
    tools=[],  
    system_prompt="You are a support agent. Confirm what the customer told you in one  
line.",  
    middleware=[  
        PIIMiddleware("credit_card", strategy="block")  
    ],  
)  
  
try:  
    strict.invoke({"messages": [{"role": "user", "content": "my card is 4111 1111 1111  
1111"}]})  
except PIIDetectionError:  
    print("Blocked, ask the customer never to send card numbers in chat.")
```

```
"my card is 4111 1111 1111 1111"  
|  
PIIMiddleware (block)  
|  
credit card detected  
|  
PIIDetectionError raised ---> model is NEVER called  
|  
your except block  
|  
"Blocked, ask the customer never to send card numbers in chat."
```

In a real product the `except` block is where your business logic goes: show a polite message, log the attempt, notify someone. Printing is the placeholder.

6 Custom state: when messages are not enough

An agent's state holds one thing that matters to it: the list of messages. That is enough for the agent to hold a conversation. It is not enough for your application.

Your application wants to know things like:

- How many tools has this agent used so far?
- Has a refund already been started?
- Which department is handling this case?
- Has the customer been verified?

None of that is a message. So you add your own fields to the state, sitting alongside the messages, and let middleware keep them up to date.

THE DEFAULT STATE

```
{
  "messages": [ ... ]
}
```

enough to chat

YOUR STATE

```
{
  "messages": [ ... ],
  "tool_uses": 3,
  "refund_started": true
}
```

enough to run a business on

and a middleware watches the run and keeps those two keys current.

DOES THIS EXTRA STATE COST TOKENS?

Many people asked this in the chat, worried it would eat the context window. It does not. Adding keys to the agent state does not send them to the model. **Only the messages travel to the LLM.** Your custom fields are for your code. A student described it as audit information, and the trainer agreed.

Declaring the shape

```
from langchain.agents.middleware.types import AgentMiddleware, AgentState

class SupportState(AgentState):
    """The usual messages, plus two keys of our own."""

    tool_uses: int          # how many tool calls have happened so far
    refund_started: bool    # True once start_refund has been called
```

Inheriting from `AgentState` keeps everything the agent already needs, including `messages`. You are extending the state, not replacing it.

7 A middleware class that updates the state

So far every custom middleware was a function with a decorator. This one is a class, which is the other way to write middleware, and the natural choice when it needs to carry a schema.

custom_state.py

```
@tool
def order_status(order_id: str) -> str:
    """Get the status of an order using its id."""
    return f"{order_id.upper()}: packed, ships tomorrow"


@tool
def delivery_estimate(pin_code: str) -> str:
    """Estimate delivery days for an Indian pin code."""
    return "5 days"


@tool
def start_refund(order_id: str, reason: str) -> str:
    """Start a refund for an order."""
    return f"Refund started for {order_id.upper()}, reason: {reason}"


class TrackWork(AgentMiddleware):
    """Counts tool calls and notices when a refund has been started."""

    state_schema = SupportState  # tell LangChain my state looks like SupportState

    # runs after the model, before any requested tool executes
    def after_model(self, state, runtime):
        last = state["messages"][-1]
        calls = getattr(last, "tool_calls", []) or []
        if not calls:
            return None

        update = {"tool_uses": state.get("tool_uses", 0) + len(calls)}

        if any(call["name"] == "start_refund" for call in calls):
            update["refund_started"] = True

        print(f"    [state] tool_uses={update['tool_uses']} "
              f"refund_started={update.get('refund_started', state.get('refund_started', False))}")

        return update
```

The three lines doing the real work

- `state_schema = SupportState` is what connects this middleware to your custom fields. Leave it out and the state has no `tool_uses` and no `refund_started`. They would simply not exist.
- `def after_model(self, state, runtime)` has no decorator. Inside an `AgentMiddleware` class, LangChain recognises the method by its name, so `after_model` is automatically the after-model hook.
- `return update` is how the state changes. The middleware never edits the state directly. It returns a dictionary of the keys it wants changed, and LangChain applies it.

WHY AFTER_MODEL, AND WHY COUNT LEN(CALLS)

The model decides a tool is needed. Only then does the tool run. So `after_model` is the moment the model has announced its intention and nothing has executed yet. That is when you record it.

And the increment is `len(calls)`, not `+ 1`, because last session showed a model asking for several tools in a single turn.

A prompt that reads the state

```
@dynamic_prompt
def prompt_with_budget(request):
    used = request.state.get("tool_uses", 0)

    prompt = "You are a support agent. Look things up with the tools, never guess."
    if used >= 3:
        prompt += " You have already used three tools, answer now with what you have."
    return prompt
```

This is `@dynamic_prompt` from last session, but reading `request.state` instead of a customer context. Once three tools have been used, the prompt tells the model to stop reaching for more and answer. A simple budget.

Wiring it up and running it

```
agent = create_agent(
    model="gpt-4o-mini",
    tools=[order_status, delivery_estimate, start_refund],
    middleware=[TrackWork(), prompt_with_budget],
)

result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": "Where is ORD-1002, when does it reach pin 560034, "
        "and refund it because it is too late.",
    }],
    "tool_uses": 0,          # your fields need starting values
    "refund_started": False,
})

print("Answer:", result["messages"][-1].content)
print("tool_uses      :", result["tool_uses"])
print("refund_started :", result["refund_started"])

if result["refund_started"]:
    print("Refund has been started, notify the finance team and support team")
```

```
[state] tool_uses=3 refund_started=True
```

Answer: Your order is packed and ships tomorrow, it should reach
560034 in 5 days, and a refund has been started.

```
tool_uses      : 3
refund_started : True
Refund has been started, notify the finance team and support team
```

the last line is ordinary Python reading your custom state.
that is the point: your application can now act on what the agent did.

HONEST STATUS FROM THE LIVE CLASS

The first run failed because the messages and the starting state values had been put inside `create_agent` instead of `agent.invoke`. The trainer called it a silly mistake from being carried away with the lines of code, and a student pointed it out.

The rule is worth keeping: `create_agent` gets what the agent *is*, meaning model, tools, middleware, prompt. `invoke` gets what this *run* is, meaning messages and state. Also, if you do not pass starting values for your custom keys, reading them fails because they do not exist yet.

8 Middleware, in one page

The first half ended with a quick quiz to the room. Here are the questions with the answers.

Question	Answer	Built in or custom
A layer around an agent that controls what happens before, during and after it runs	Middleware	Concept
Protect sensitive data before it reaches the model	<code>PIIMiddleware</code>	Built in
Choose the right model for the question	Model routing, on <code>@wrap_model_call</code>	Custom
Change the instructions based on context or state	<code>@dynamic_prompt</code>	Custom
Retry a tool that is not responding	<code>ToolRetryMiddleware</code>	Built in
Manage a long conversation	<code>SummarizationMiddleware</code>	Built in
Control which tools are offered	Dynamic tool selection, on <code>@wrap_model_call</code>	Custom
Does a <code>block</code> strategy ever reach the model?	Never	

WHY MIDDLEWARE EXISTS, IN ONE SENTENCE

So the agent is not responsible for everything. The agent does its actual work, calling the model and the tools, and anything around that is written separately. Separation of concerns, the same principle you already know from ordinary backend code.

9 MCP: the problem before the protocol

Here Akshay Agarwal took over. He framed the topic honestly: MCP was released by Anthropic in late 2024 and became widely used during 2025, so nobody has ten years of experience in it, and he said he would say so openly if he did not know an answer.

The session is theory only. No MCP code yet, deliberately, so the "why" is settled first.

THE RECAP HE ASKED THE ROOM FOR

Models, tool calling, custom tools, agents with `create_agent` , middleware, prompt templates, output parsers, LCEL, image generation, memory, and the knowledge cutoff that makes tools necessary in the first place. Everything in MCP builds on tools, so if tools are clear, this will be.

The pain point

You have built an LLM application and written four tools for it. Now a developer somewhere else needs exactly those four tools. With what you know so far, they write them again.

That is not how software normally works. The tools exist. They are just not **exposed**. They live inside your program on your machine.

DEVELOPER A

DEVELOPER B

+-----+

| LLM |

| + get_tweets() |

| + create_tweet() |

| + create_ticket() |

+-----+

+-----+

| LLM |

| + get_tweets() | written again

| + create_tweet() | written again

| + create_ticket() | written again

+-----+

the same tools, built twice, maintained twice.

nothing lets B use what A already built.

We have solved this before, with APIs

When you build an app like Uber, you do not build your own maps. You call Google Maps' APIs. Somebody else built it, validated it and maintains it, and you use it. That works because of a shared protocol, HTTP, with conventions everyone follows: what a GET means, what a 200 means, what a 404 means. Any program anywhere can call any API that follows them.

MCP is the same idea for tools. Tools need to be exposed to the world in a standard way so other programs can use them.

10

Why not just wrap every API in a tool

This is the obvious objection and it was raised immediately. We already know how to write a tool. A tool is a Python function, and a Python function can call any API. The APIs already handle authentication, authorisation and roles. So why a new protocol?

It is a fair question, and the answer comes from what happens over time.

YOUR APPLICATION

```
|
one tool that calls X (Twitter) APIs directly
|
+--> POST /v1/create_tweet
+--> GET  /v1/get_tweet
+--> PUT  /v1/update_tweet
```

then, one by one:

PROBLEM 1 X moves to /v2/create_tweet
 -> you update your code

PROBLEM 2 the request or response shape of create_tweet changes
 -> you update your code again

PROBLEM 3 you add GitHub as well
 different auth, different status codes (one returns 200
 for a create, the other correctly returns 201), different
 pagination, different everything
 -> you write different handling logic for every service

every service you add is another set of rules to learn and maintain.

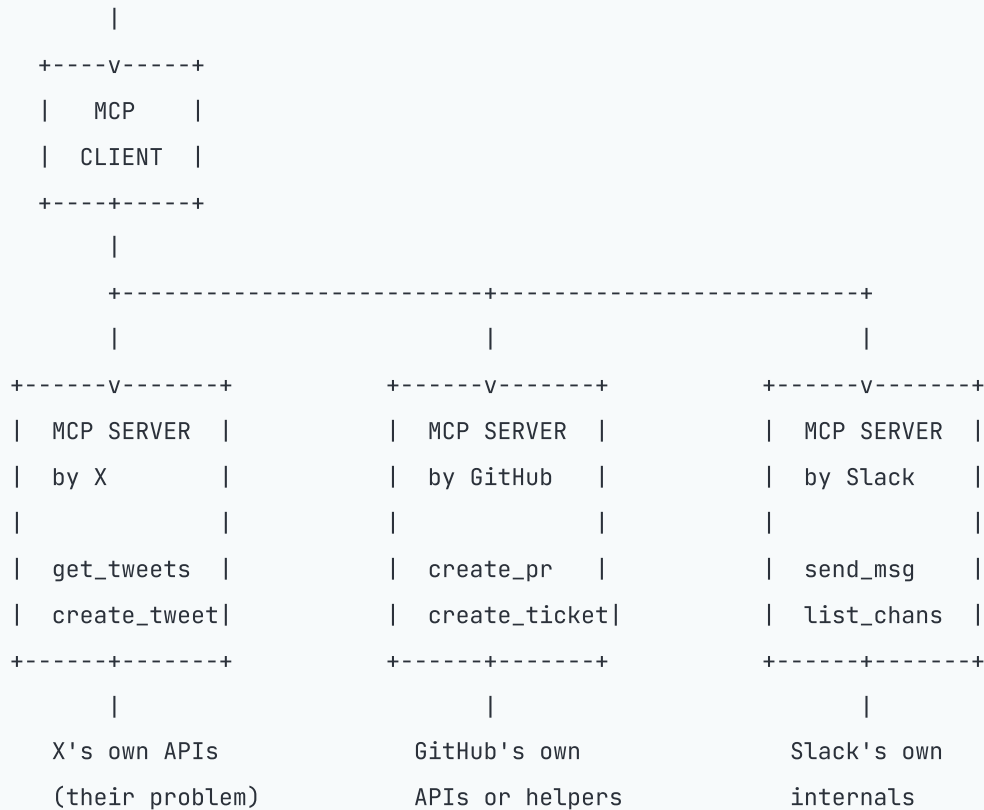
THE LINE TO REMEMBER

HTTP standardised how applications call APIs. Before MCP there was no standard for how an AI application calls tools. **MCP standardises the tool call.** That was repeated as the single sentence to take away from this half of the session.

11 What MCP actually changes

Instead of your code talking to Twitter's API and GitHub's API directly, each company puts a layer in front of its own service. That layer is an **MCP server**, and it offers tools. Your application talks to those servers through an **MCP client**.

YOUR APPLICATION



you talk to tools, in one standard way.

what sits behind a tool is the owning company's responsibility.

- **Endpoint changes?** Not your problem. You call a tool, not an endpoint. The company updates the tool behind it.
- **Response shape changes?** The tool absorbs it.
- **Different services behave differently?** Every server speaks the same protocol, so your client handles them all the same way.

WHO BUILDS THE SERVER

A good question from the room. You do not build an MCP server for X or GitHub. **They do**, exactly as it has always been their responsibility to publish their APIs. You integrate it. You would build your own server when you want to expose your own application's functionality, including internally.

Also answered: behind a tool, the company may call its own APIs, or it may call its internal helper functions directly. That is their choice. You never see it.

IT IS NOT FREE OF COST

Asked directly: will companies charge for their MCP servers? They can, the same way APIs are often paid. Integrating third-party functionality has always had a price, and MCP does not change that. It changes how you integrate, not whether you pay.

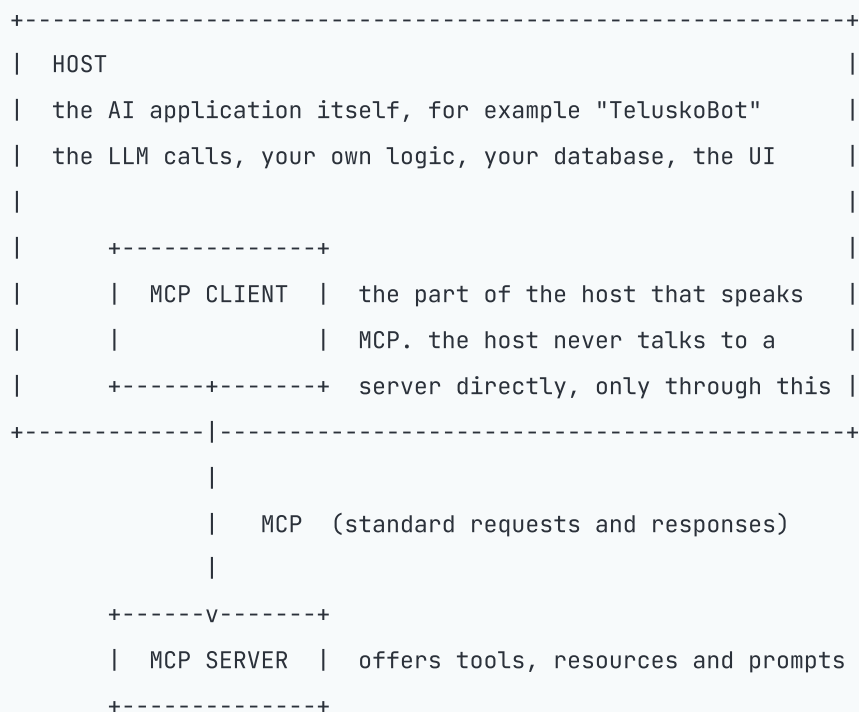
12 The architecture: host, client, server

MCP uses three words precisely, and they are worth getting right now.

THE USER

| "review the latest PR"

v



request goes host -> client -> server

response goes server -> client -> host -> user

the user knows none of this. they are just talking to TeluskoBot.

Term	What it is
Host	The AI application the user interacts with. It contains the LLM calls and your own logic.
Client	The component inside the host that connects to an MCP server. The host goes through it, never around it.
Server	The program that exposes capabilities. Built by whoever owns those capabilities.

A student compared this to Java's RMI, remote method invocation, and the comparison was accepted as similar in spirit: calling functionality that lives elsewhere as if it were local.

Official specification: modelcontextprotocol.io

13 The three primitives: tools, resources, prompts

A server can offer three kinds of things. Together they are called **MCP primitives**. All three are optional, a server can offer any combination.

Tools

Functions the server exposes and the client can call, to perform an operation or fetch information. You already know these well. They are the same tools as before, now reachable over a standard protocol.

Resources

Mostly static data that the server shares: a configuration file, a README, documentation, a database schema. Things that do not change often. Useful when the model needs context rather than an action. If you want to work with a database, being able to read its schema is more useful than another tool.

Prompts

Ready-made prompt templates maintained by the server's owner. The example given in class shows exactly why these exist.

a tech lead types into TeluskoBot:

```
"review the PR"                                <-- vague. it means nothing specific.
```

the host asks GitHub's MCP server for its PR review prompt, and gets:

```
"Review this pull request for:
- readability
- correctness
- whether it is bug free
- performance
and list concrete issues with line references."
```

that better prompt is what actually goes to the model.
the lead typed three words. the review is thorough anyway.

This is prompt templates from session 3, except the template is maintained by the company that knows its domain best, and it is published through the server.

CAN I IGNORE THE SERVER'S PROMPTS AND WRITE MY OWN?

Yes. Asked directly, and the answer was that nothing is mandatory. If a server offers five hundred tools you do not use all five hundred. If it offers prompts and resources, you use the ones that help and ignore the rest.

14 The standard methods every server speaks

This is what "standardised" means concretely. Whether the server belongs to GitHub, X or you, the client uses the same method names for the same things.

TOOLS	tools/list	what tools do you have?
	tools/call	run this one, with these arguments
RESOURCES	resources/list	what resources do you have?
	resources/read	give me the contents of this one
	resources/subscribe	tell me when this one changes
	resources/unsubscribe	stop telling me
PROMPTS	prompts/list	what prompt templates do you have?
	prompts/get	give me this one, filled in

every MCP server, from every company, answers these the same way.

The class wrote these loosely on the board. The names above are the exact method names from the official specification, which you will see in real request payloads in session 8.

WHAT IF A COMPANY RENAMES A TOOL?

Say GitHub renames a tool from `get` to `fetch`. Your code does not break in the way a renamed API endpoint would.

`tools/list` returns the new name along with its description. And remember that you are not calling tools by hand, the **LLM** picks them, using the name and the description, exactly as in session 4. The description still says it gets ticket data, so the model still picks it. The trainer did say you may occasionally need to adjust something, so it is not entirely free, but it is far more forgiving than a hard-coded endpoint.

SUBSCRIBE EXISTS BECAUSE RESOURCES DO CHANGE

Resources are "mostly static", not frozen. Documentation gets updated when features ship.

`resources/subscribe` means the server notifies you when a resource you care about has changed, instead of you polling for it.

15 The data layer: JSON-RPC 2.0

The session closed on how MCP actually carries messages. MCP's data layer is built on **JSON-RPC 2.0**. Two ideas combined.

JSON

JavaScript Object Notation. Key and value pairs in curly braces, lists in square brackets, very close to a Python dictionary. It is the standard way data moves between systems today, and XML has largely given way to it. If JSON is new to you, a short tutorial is enough; it is covered in the Python gaps recordings from the start of the course.

RPC

Remote Procedure Call. You are working on your machine and you use a function that lives on another machine, without it feeling like you are working on another machine. It feels local.

The analogy that came up was VirtualBox, and the instructor was careful to say it is only roughly right, because a virtual machine is on your computer while RPC reaches a genuinely different one. The idea that matters: your code calls a procedure, and where it actually runs is someone else's concern. That is also what makes distributed systems possible.

Put together

JSON gives the shape of the message. RPC gives the idea of calling a remote function with it.

THE REQUEST

```
{
  "jsonrpc": "2.0",
  "method": "add",
  "params": [2, 3],
  "id": 1
}
```

THE RESPONSE

```
{
  "jsonrpc": "2.0",
  "result": 5,
  "id": 1
}
```

jsonrpc which version of the protocol, always "2.0"

method the procedure you want run

params its arguments

id a number you choose. the response carries the same id,
 so when several requests are in flight you can tell
 which answer belongs to which question

the response has no method and no params. it has a result instead.
(or an "error" object, when something went wrong)

WHY THIS MATTERS FOR EVERYTHING IN SESSION 8

The MCP client builds requests in exactly this shape, and the server replies in exactly this shape. The `method` field is where names like `tools/list` and `tools/call` from section 14 go. Once you have seen one request and one response, every MCP message will read naturally.

JSON-RPC 2.0 specification: jsonrpc.org/specification

The instructor stopped here because the three hours were up, and said this communication part would be repeated at the start of the next class. He also said plainly that MCP will take as many sessions as it needs, even if the course runs a few weeks longer.

16 Clarifications from the Q&A

On middleware

- **Is middleware the same as guardrails?** No. Guardrails, with input guardrails and output guardrails, are a separate topic coming later. PII middleware overlaps with the idea, but middleware is the general mechanism.
- **Does the custom state go to the model?** No. Only messages travel to the LLM. Custom keys do not consume the context window.
- **Can custom state be used in production?** Yes.
- **Can I notify someone when PII is blocked?** Yes. That is ordinary business logic inside your error handling.

On MCP

- **Can the LLM call an API directly?** No. The LLM can call tools, and tools can call APIs.
- **Do tools inside an MCP server need an LLM?** No. A tool is plain Python, Java or whatever language the server is written in. The LLM is only involved on the client side, where it decides which tool to use. You choose any model for that.
- **What if a company has no MCP server?** Then you are back to writing your own tools that call their APIs, which is what you did before MCP.
- **Is an MCP client the LLM?** No. It is closer to the tool calling machinery. The client reads the server configuration and talks to the servers; the LLM decides what to call.
- **Can multiple clients map to multiple servers?** Yes. And in practice all the servers an application uses are declared together in one configuration, which session 8 shows.
- **How is security handled?** The same way as with APIs: credentials, secret keys and tokens passed within a session so the server can check what the caller is allowed to do.
- **Is MCP stateless?** It was mentioned in class that recent revisions of the protocol have moved towards stateless operation, while still allowing sessions where needed. Check the current specification for the details, as this area is still evolving.

- **Can MCP be used between my own microservices?** Yes. A student building a separate chatbot service that must reach existing microservices with REST APIs asked whether MCP fits. It does, and the instructor agreed that was exactly the right use.
- **Do I need MCP if my system connects to nothing external?** Deferred to the practical sessions, with the hint that exposing your own tools through your own server can still make sense.
- **Can MCP help reduce tokens?** Yes, by offering the model only the tools it needs, which ties back to dynamic tool selection in session 6.
- **Apps talk to APIs. Do agents talk to MCP?** The instructor's preferred phrasing was that the **LLM** needs MCP, since an agent is not an LLM.

17 Key takeaways

- **Personal data does not help the model answer**, so strip it before it leaves your system. `PIIMiddleware` does this with no code of your own.
- **Four strategies:** redact replaces with a label, mask keeps a hint, hash gives a stable stand-in, block stops the request before the model is ever called.
- **Where no built-in detector fits, write one.** Return `PIIMatch` objects with type, value, start and end.
- **Catch `PIIDetectionError`** when you use `block`, and put your real handling there.
- **Messages are enough to chat, not to run a business.** Extend `AgentState` with your own keys.
- **`state_schema` connects a middleware to those keys**, and a middleware changes state by returning a dictionary, never by editing it.
- **In a middleware class, method names are the hooks.** No decorator needed.
- **Custom state does not travel to the LLM.** Only messages do.
- **`create_agent` gets what the agent is; `invoke` gets what the run is.**
- **Tools you write are not reusable by anyone else until they are exposed.**
- **Wrapping APIs in tools breaks over time:** versions change, shapes change, and every service behaves differently.
- **MCP standardises the tool call**, the way HTTP standardised the API call.
- **Host, client, server.** The host is your app, the client speaks MCP on its behalf, the server is built by whoever owns the capability.
- **Three primitives:** tools to act, resources for mostly static context, prompts for templates maintained by the domain owner.
- **Every server answers the same methods**, `tools/list`, `tools/call`, `resources/read`, `prompts/get` and so on.
- **MCP messages are JSON-RPC 2.0:** `jsonrpc`, `method`, `params`, `id` going out; `result` or `error` with the same `id` coming back.

COMING NEXT

MCP continues the next day. The JSON-RPC communication part gets repeated properly, then real request and response payloads for listing and calling tools, the lifecycle of a connection from initialisation onwards, and the transports that carry the messages. After the theory, code: an MCP server of your own built with FastMCP, and a LangChain client that loads its tools and hands them to a model.

Telusko | Agentic AI Engineering with Python. Notes for Session 07, recorded 12 September 2026.